

Pearls AQI Predictor

72-Hour Air Quality Forecasting — 100% serverless stack

Project Report | Author: Mahnoor Ishtiaq | 2026

1. Introduction

Karachi, Pakistan, is consistently ranked among the most polluted megacities in the world. Millions of residents are exposed daily to hazardous levels of particulate matter and toxic gases, yet there is no accessible, locally tailored, real-time AQI forecasting service with model explainability available for the public. This project addresses that gap.

The Karachi AQI Predictor is a fully serverless, automated machine learning pipeline that ingests live weather and pollutant data, trains multiple models, and serves 72-hour AQI forecasts through an interactive Streamlit dashboard backed by a FastAPI inference engine.

Key Capabilities

- 72-hour hourly AQI forecast using recursive ML inference
- Fully automated feature ingestion (every hour) and model retraining (every day)
- Interactive dashboard with real-time AQI gauge, trend charts, pollutant breakdowns
- SHAP-based model explainability to understand prediction drivers
- Hazardous AQI alerts with health guidance messages
- Zero dedicated server infrastructure — fully cloud-native and serverless

2. Problem Statement

Air Quality Index forecasting requires solving three interconnected engineering challenges simultaneously: reliable real-time data ingestion, accurate multi-step time-series prediction, and a production-ready serving layer, all without expensive infrastructure.

3. Technical Challenges & Alternatives Taken

1. Feature Store & Model Registry: Hopsworks vs. MongoDB Atlas + DagsHub

Initial Approach: Hopsworks

Hopsworks was originally planned feature store and model registry. Its Python SDK appeared well-suited for versioned feature groups and online feature serving during inference.

Alternative: MongoDB Atlas (Feature Store) + DagsHub (Model Registry)

After evaluating alternatives, the project adopted a two-component replacement that aligned perfectly with the project's serverless architecture:

- MongoDB Atlas: Free-tier cloud database used as a flexible, schema-less feature store. Documents are inserted hourly by the feature pipeline and queried by the training and inference pipelines. No SDK conflicts: standard PyMongo driver worked out of the box.
- DagsHub + MLflow: Git-native ML experiment tracker and model registry. Models are logged via the standard MLflow API and promoted to the Production stage.

2. Data API: OpenWeather vs. OpenMeteo

Initial Approach: OpenWeather API

OpenWeather was the first choice for weather and air quality data, given its widespread use in ML projects and comprehensive pollutant coverage. OpenWeather Air Pollution API required a paid subscription for historical data access beyond the most recent 24 hours. The free tier only provided current-hour readings, making it impossible to perform the 90-day historical backfill required for model training without incurring cost.

Alternative: OpenMeteo API

OpenMeteo provides both weather and air quality data through a unified, completely free API with no API key required. Critically, it supports historical data queries going back several years with hourly resolution, enabling the full backfill without cost.

- Air quality endpoint: PM2.5, PM10, NO₂, O₃, CO, SO₂
- Weather endpoint: temperature, humidity, wind speed
- Historical access: up to 3 months of hourly data per API call
- No authentication required, which reduced pipeline complexity

3. Non-Deterministic Predictions: Random Drift vs. Deterministic Inference

Issue: Predictions Changed on Every Dashboard Refresh

Early versions of the inference engine (Predict.py) used random noise to simulate drift in exogenous features (temperature, humidity, wind speed) across the 72-hour window. This caused the AQI forecast to change by up to ± 15 AQI units on every page refresh, undermining user trust and making the system non-cacheable.

Fix: Deterministic Recursive Inference

The inference engine was redesigned to hold exogenous features (weather) constant across the 72-hour window. While this is a simplification of reality, it is the correct trade-off for a deterministic, cacheable, user-trustworthy system. Lag features are propagated correctly using a deque data structure for the 24-hour sliding window. This ensured that Dashboard refreshes no longer cause forecast changes until new hourly data arrives from the feature pipeline.

4. Exploratory Data Analysis (EDA)

Data was collected using the Open-Meteo API, including hourly pollutant and weather data for Karachi (Lat: 24.8607, Lon: 67.0011). EDA was conducted on 90 days of historical data to understand AQI trends, distributions, correlations, and temporal behavior before model training.

Collected Features:

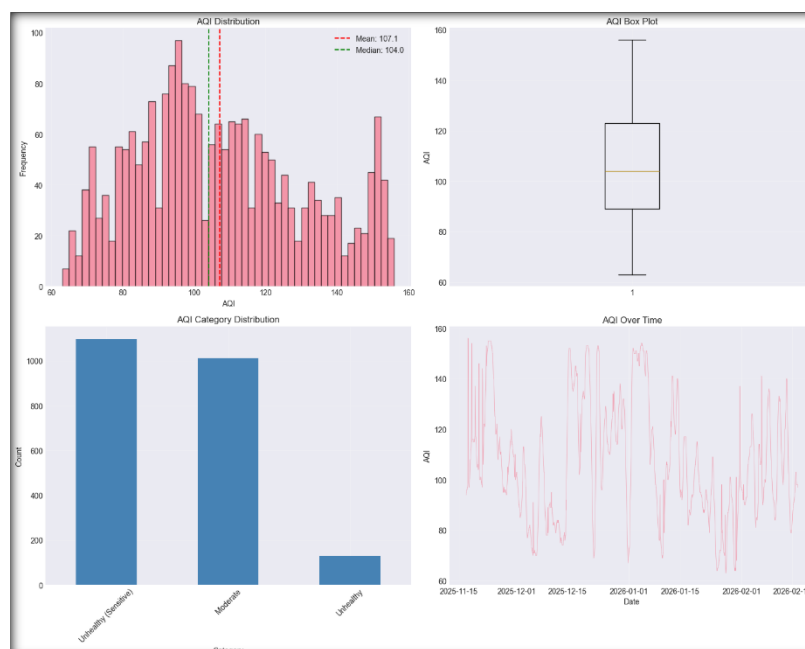
- **Pollutants:** PM2.5, PM10, CO, NO₂, O₃, SO₂
- **Weather:** Temperature, humidity, wind speed
- **Time:** Hour, day of week, month
- **Derived Features:**
 - AQI lag features (1-hour, 24-hour)
 - AQI change rate (1-hour, 24-hour)
 - Time-based encodings

Feature Selection Approach:

- Correlation analysis identified PM2.5 and PM10 as strong pollutant predictors.
- Lagged AQI (especially 1-hour lag) showed the highest correlation with target AQI.
- Weather variables (temperature, humidity, wind speed) showed moderate influence.
- SHAP analysis validated feature importance for interpretability.
- Final engineered features were stored in MongoDB Atlas for model training.

Key Insights:

- AQI spikes are strongly influenced by PM2.5 concentration and short-term temporal dependence.
- Low wind speed slightly increases AQI levels.
- Lag features significantly improve predictive accuracy in time-series forecasting.



5. System Architecture

The system follows a clean data-flow architecture with four independently deployable components connected through cloud storage layers.

Component	Technology & Role
Data Source	OpenMeteo API - free hourly weather + pollutant data, historical backfill
Feature Store	MongoDB Atlas - cloud document store for processed feature vectors
Feature Pipeline	Python + GitHub Actions - hourly automated feature ingestion
Training Pipeline	Scikit-learn + MLflow + GitHub Actions - daily model training
Model Registry	DagsHub - MLflow remote server, Production model staging
Inference Engine	Predict.py - recursive 72-hour deterministic forecasting
API Backend	FastAPI - async REST API serving forecast data
Dashboard	Streamlit + Plotly - interactive real-time visualization
Explainability	SHAP + Jupyter Notebook - feature importance analysis

6. Feature Pipeline

The feature pipeline (**feature_pipeline.py**) is the data backbone of the entire system. It runs every hour via GitHub Actions, fetches the latest weather and pollutant readings, engineers all model-ready features, and writes a new document to MongoDB Atlas.

Collected Features

Feature Category	Features
Pollutants	PM2.5, PM10, NO ₂ , O ₃ , CO, SO ₂
Weather	Temperature (°C), Humidity (%), Wind Speed (m/s)
Time-Based	hour, day, month, day_of_week
Lag Features	aqi_lag_1 (1 hour prior), aqi_lag_24 (24 hours prior)
Derived Features	aqi_change_1h, aqi_change_24h (rate of change)
Target	target_aqi (next hour AQI — for training only)

Historical Backfill

backfill.py runs the feature pipeline across the past 90 days of historical data, building the comprehensive training dataset. Features computed during backfill are stored with `dataset_type: 'historical'` in MongoDB to separate them from live online features.

- 90 days × 24 hours = ~2,160 training samples minimum
- Each sample contains 17 features + 1 target
- Backfill is idempotent — safe to re-run without duplicating data

7. Model Development & Evaluation

Models Trained

Three scikit-learn models were trained and compared to identify the best performer:

- **Gradient Boosting Regressor:** ensemble boosting, typically best accuracy for tabular time-series
- **Random Forest Regressor:** bagging ensemble, robust to noise and non-linearity
- **Ridge Regression:** linear baseline, fast and interpretable

Evaluation Metrics

Metric	Description
RMSE	Root Mean Squared Error: penalizes large prediction errors
MAE	Mean Absolute Error: average absolute AQI prediction error
R ²	Coefficient of Determination: proportion of variance explained

Training Process

- Historical features fetched from MongoDB Atlas (dataset_type: 'historical').
- Time-based train/test split (80% train, 20% test — respects temporal ordering).
- All three models trained and evaluated on identical splits.
- Best model (lowest RMSE) promoted to DagsHub Production registry.
- All runs logged to MLflow: parameters, metrics, model artifacts.

Performance Evaluation

Model	RMSE	MAE	R ²
GradientBoosting	1.87	1.06	0.987
Ridge Regression	2.13	0.98	0.983
RandomForest	4.18	2.54	0.933

AQI Prediction

Days	AQI	Category
1	85	Moderate
2	114	Unhealthy (Sensitive)
3	148	Unhealthy (Sensitive)

8. Inference Engine - Predict.py

The inference engine generates a 72-hour forecast through recursive single-step prediction. At each step, the model predicts the next hour's AQI, which is then fed back as the lag feature for the subsequent step.

Algorithm

- Load last 24 hours of real data from MongoDB Atlas (online features)
- Initialize 24-element deque with actual AQI values for lag tracking
- For each of 72 future hours:
 - Update temporal features (hour, day, month, day_of_week)
 - Compute lag features from deque (lag_1, lag_24, change_1h, change_24h)
 - Run model. Predict() on assembled feature vector
 - Append prediction to deque for next iteration
- Return list of 72 {timestamp, predicted_aqi} objects

9. CI/CD Pipeline - GitHub Actions

Both pipelines run automatically via GitHub Actions with no manual intervention required after initial setup.

Workflow	Schedule & Purpose
feature_pipeline.yml	Every hour (cron: '0 * * * *') — fetches live data, computes features, writes to MongoDB
train.yml	Daily at 2 AM UTC (cron: '0 2 * * *') — trains models, logs to MLflow, promotes best model

10. Web Application Dashboard

The dashboard is split into a FastAPI backend (serving predictions) and a Streamlit frontend (visualizing results). All charts use Plotly for interactivity and consistent theming.

Dashboard Tabs

Tab	Content
Overview & Forecast	Current AQI gauge, weather metrics, 72-hour forecast chart with AQI zones, pollutant trends
Historical Analysis	7–30 day AQI trend, 24-hour rolling average, hourly pattern, AQI category distribution, correlation matrix
Model Performance	RMSE / MAE / R ² comparison across all three models, training metadata, best model highlight
Data Explorer	Raw tabular data (historical + forecast), CSV download buttons

Key UI Features

- AQI gauge with colour-coded zones (Good → Hazardous)
- AQI zone background overlays on all time-series charts
- Peak and valley annotations on forecast chart
- 24-hour rolling average toggle on historical chart

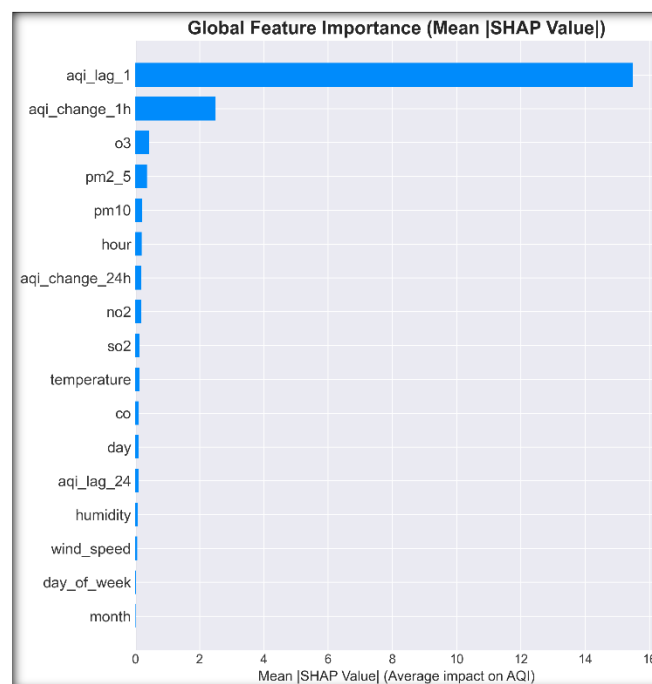
- Health message banner (colour-coded: green / yellow / red)
- Automatic alert banner when forecast exceeds AQI 150
- Theme selector (5 Plotly themes)
- Adjustable historical data range (1–30 days)
- CSV download for both forecast and historical data

11. SHAP Explainability & Advanced Analytics

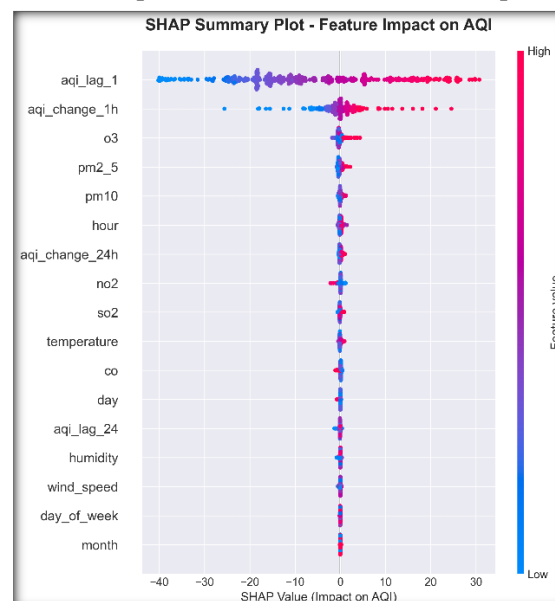
SHAP (SHapley Additive exPlanations) analysis is performed in a dedicated Jupyter notebook (notebooks/SHAP_Explainability.ipynb) to explain both global model behavior and individual predictions.

Visualizations Generated

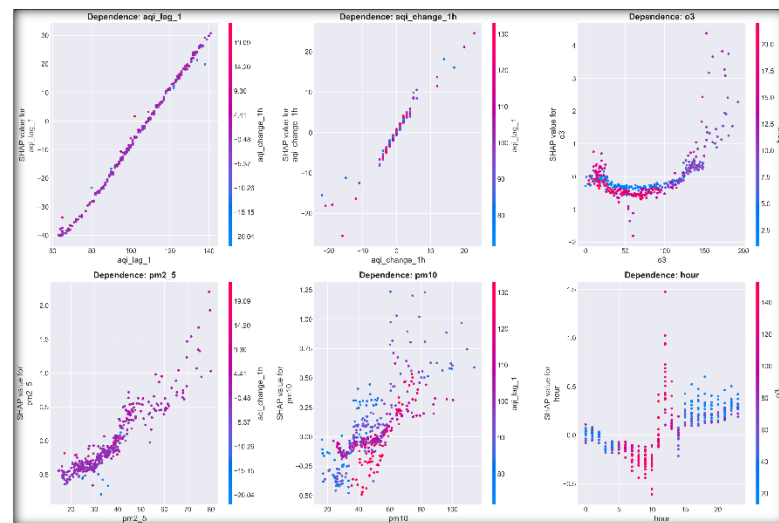
Feature importance bar chart



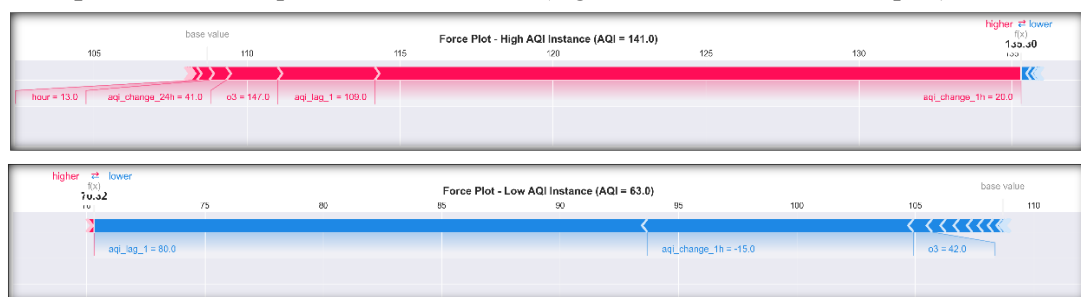
Beeswarm summary plot: feature impact distribution across all samples



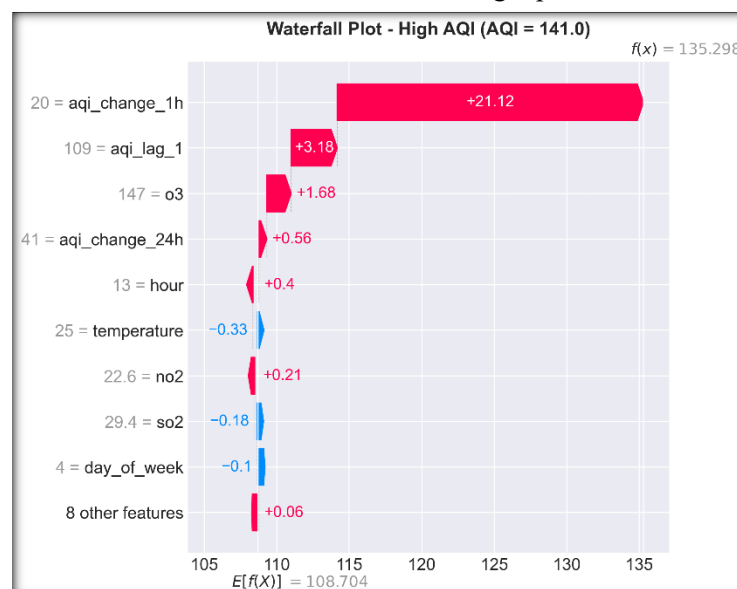
- **Dependence plots:** relationship between top features and AQI

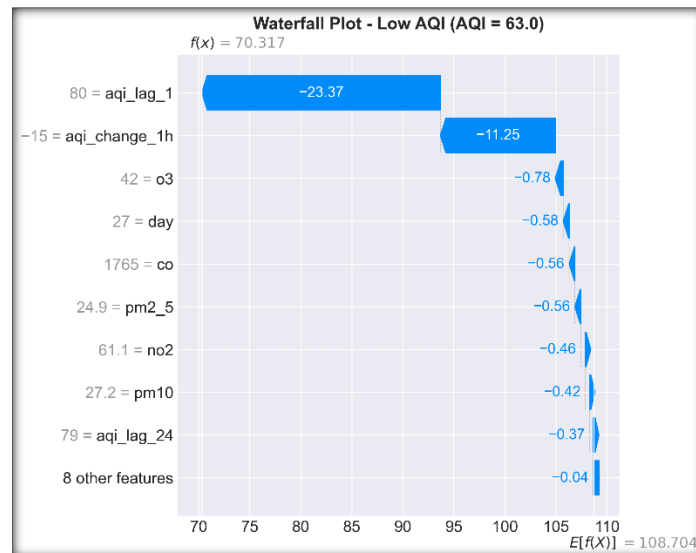


- **Force plots:** individual prediction breakdown (high AQI and low AQI examples)

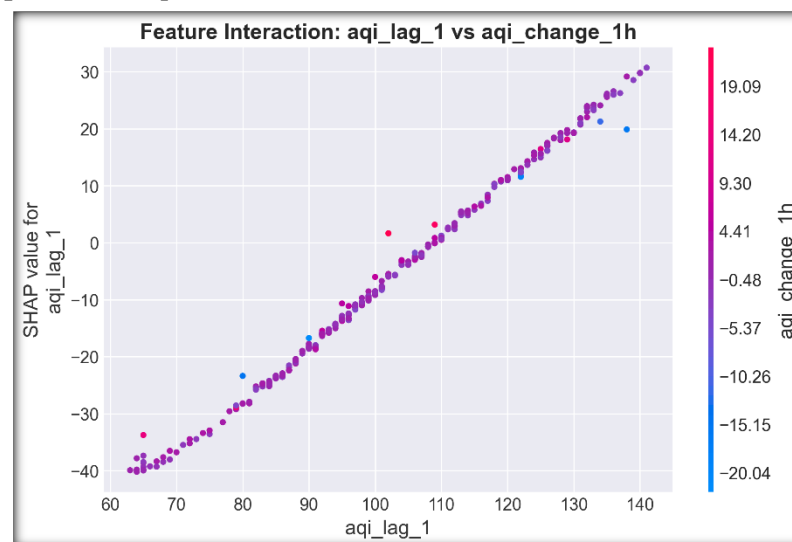


- **Waterfall plots:** cumulative feature contribution to a single prediction

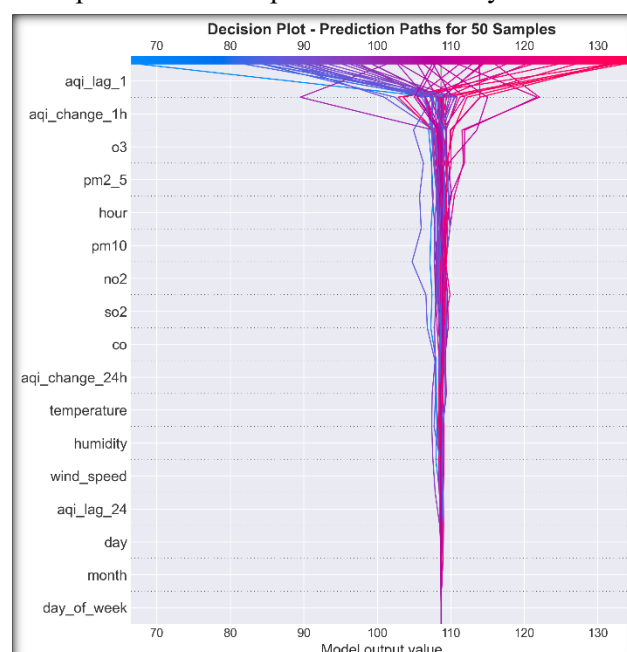




- **Interaction plots:** how two features modulate each other



- **Decision plot:** prediction paths for 50 samples simultaneously



12. Evaluation & Limitations

1. System Strengths

- Fully automated end-to-end pipeline with zero manual intervention
- Serverless architecture, no infrastructure cost or management
- Deterministic inference, reproducible, cacheable, trustworthy predictions
- Production-grade error handling at every pipeline stage
- Comprehensive model explainability via SHAP
- Interactive, professional-quality Streamlit dashboard

2. Known Limitations

- No uncertainty quantification, point estimates only, no confidence intervals
- Trained specifically for Karachi, other cities require a new pipeline run
- Sudden pollution spikes (industrial accidents, crop burning) cannot be predicted
- Dependent on OpenMeteo API uptime and data availability
- No deep learning models deployed, LSTM / Transformer models are future work

13. Future Improvements

Improvement	Impact
Quantile regression for confidence intervals	Add uncertainty bands to forecasts
LSTM / Transformer models for long-range forecasting	Better capture of seasonal and long-term trends
Multi-city AQI prediction	Expand system to Lahore, Islamabad, other cities
Email / SMS alerts for hazardous AQI levels	Proactive user notification system
Real-time WebSocket dashboard updates	Eliminate manual refresh requirement
Concept drift monitoring	Detect when model needs retraining due to data distribution shift

14. Conclusion

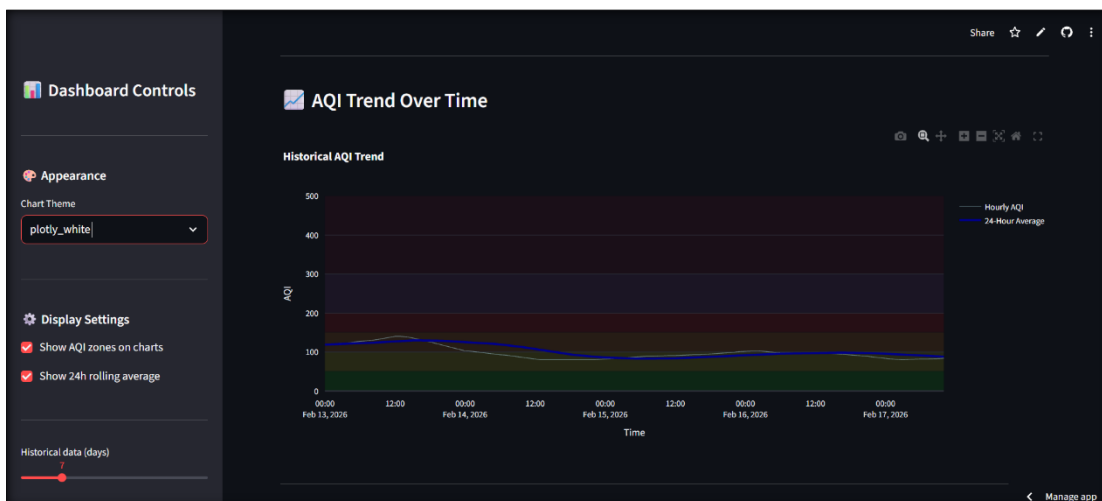
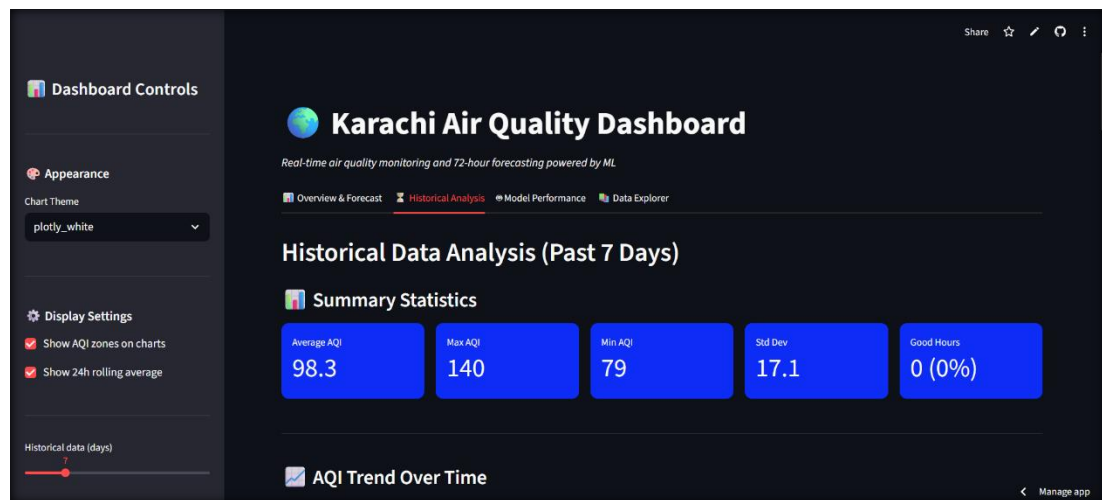
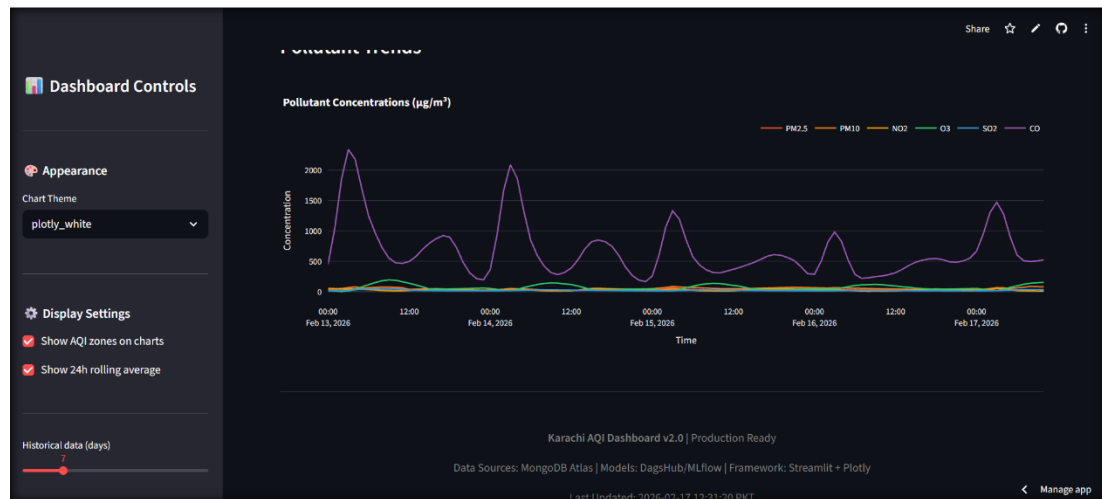
The Karachi AQI Predictor successfully delivers a production-grade, serverless, end-to-end machine learning pipeline for 72-hour air quality forecasting. The system ingests live data hourly, retrains daily, and serves interactive forecasts through a professional dashboard, all without dedicated server infrastructure.

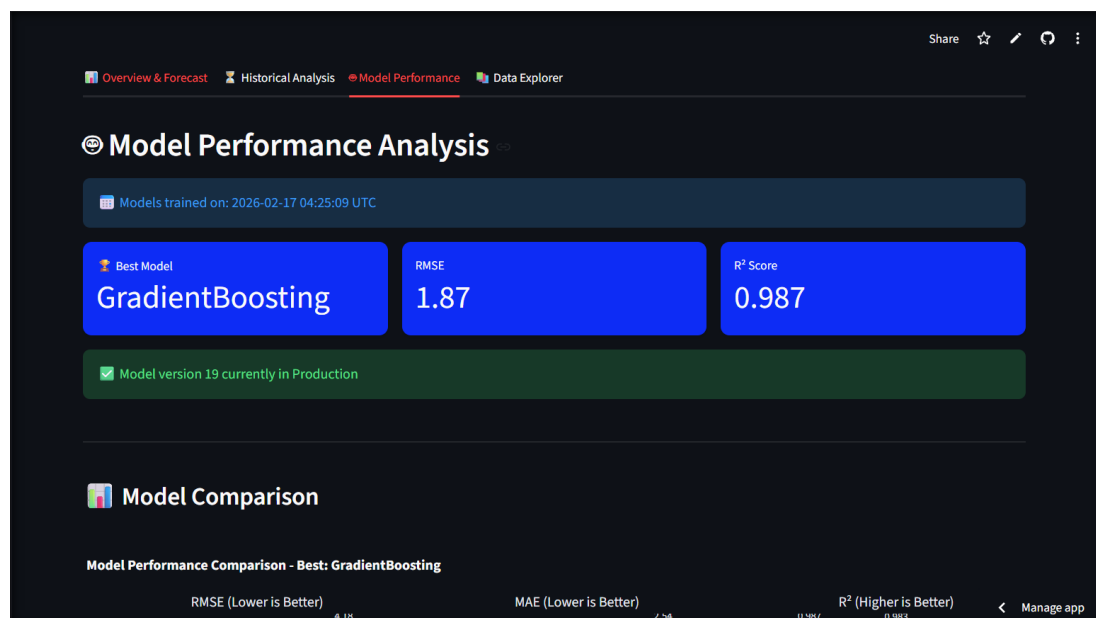
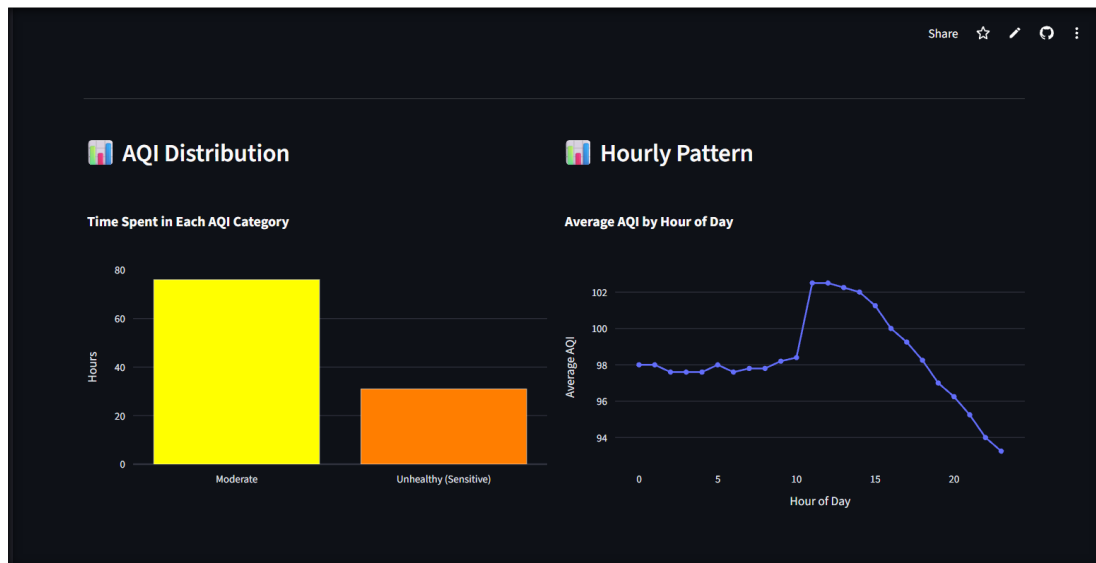
The engineering journey required overcoming three significant technical challenges: replacing Hopworks with MongoDB Atlas + DagsHub, switching from OpenWeather to OpenMeteo, and redesigning the inference engine for deterministic reproducibility. Each obstacle led to a more robust and maintainable solution than the originally planned approach.

The result is a system that is not only technically sound but genuinely useful, providing the residents of Karachi with actionable, explainable, real-time air quality intelligence.

15. Dashboard Screenshots







Share ☆ ↗ ↺ ⋮

Detailed Metrics Table

	Model	RMSE	MAE	R ²
0	GradientBoosting	1.87	1.06	0.987
1	Ridge	2.13	0.98	0.983
2	RandomForest	4.18	2.54	0.933

Model Interpretation

What do these metrics mean?

- **RMSE (Root Mean Square Error):** Average prediction error in AQI units. Lower is better.
- **MAE (Mean Absolute Error):** Average absolute difference between predicted and actual AQI. Lower is better.
- **R² Score:** How well the model explains variance in AQI (0-1 scale). Higher is better.

Model Characteristics:

- **GradientBoosting:** Ensemble method, typically best accuracy
- **RandomForest:** Robust, handles non-linearity well
- **Ridge:** Linear model, fast and interpretable

Share ☆ ↗ ↺ ⋮

 Overview & Forecast  Historical Analysis  Model Performance  Data Explorer

Raw Data Explorer

Historical Data (Past 7 Days)

Total Records: 107

Date Range: 2026-02-13 00:00:00+00:00 to 2026-02-17 10:00:00+00:00

Columns: timestamp, pm2_5, pm10, no2, o3, co, so2, aqi, city, temperature, humidity, wind_speed, hour, day, month, da, aqi_change_1h, aqi_change_24h, target_aqi, dataset_type, schema_version

	timestamp	pm2_5	pm10	no2	o3	co	so2	aqi	city	temperature	humidity	wind_speed	hour	day	month	da
0	2026-02-13 00:00:00+00:00	38.1	43.1	53.1	12	449	14.8	119	Karachi	18.6	86	4.2	0	13	2	
1	2026-02-13 01:00:00+00:00	39.3	44.8	50	7	1047	15.8	120	Karachi	18.1	87	4.1	1	13	2	
2	2026-02-13 02:00:00+00:00	44.6	50.6	45.7	1	1859	17.4	120	Karachi	17.6	89	4.6	2	13	2	
3	2026-02-13 03:00:00+00:00	58.6	64.3	41.9	8	2339	21	121	Karachi	17.7	89	5	3	13	2	
4	2026-02-13 04:00:00+00:00	73.8	77.5	39.9	37	2181	28.3	123	Karachi	19.4	81	4.5	4	13	2	
5	2026-02-13 05:00:00+00:00	46	57.2	38.4	78	1690	37.5	126	Karachi	22.2	61	3.4	5	13	2	
6	2026-02-13 06:00:00+00:00	43.4	65	35	116	1245	44.2	127	Karachi	25.5	50	0.7	6	13		 Manage app